Especificação e Acordo de Projeto

App de Descoberta Gastronômica por Swipe · Flutter + Firebase · Benchmark de mercado, escopo técnico, cronograma e termo de compromisso

Responsável pelo projeto	Antônio Cláudio Costa Junior
Papel	Desenvolvedor mobile (Flutter): autor e executor do projeto
Tipo de documento	Acordo interno de projeto (escopo, arquitetura, cronograma e compromisso de entrega)
Versão	1.1 (24/09/2026): inclui a seção 6, Cronograma e organização do time
Status	Rascunho para assinatura / aprovação

1. Dor a ser sanada e objetivo do produto

Ao usar apps de delivery (iFood, Rappi, Uber Eats etc.), o usuário precisa primeiro decidir o que quer comer antes de conseguir decidir onde pedir. É nessa primeira etapa que ocorre a maior fricção: a chamada *decision fatigue*, agravada quando a decisão é em grupo ("eu não sei, você escolhe"). Os apps de delivery tradicionais assumem que essa decisão já foi tomada; eles resolvem "onde pedir", não "o que pedir".

O objetivo do produto é resolver essa etapa anterior à entrega com uma sessão curta de escolhas binárias (estilo swipe). A sessão começa ampla (ex.: salgado vs. doce) e converge progressivamente até um tipo de comida específico. Então, no mesmo fluxo, o app recomenda restaurantes reais e próximos que atendem esse tipo, prontos para o usuário abrir no delivery de preferência.

2. Benchmark de concorrentes diretos

Levantamento de apps já publicados que atacam o mesmo problema (decidir o que comer via swipe, sozinho ou em grupo). Nenhum deles combina as duas etapas, convergência de gosto e descoberta de restaurante real, no mesmo fluxo; cada um resolve só uma metade.

App	Mecânica	Prós	Contras
WhatToEat — Food Picker	Swipe individual em pratos, com filtros de dieta, orçamento e tipo de cozinha.	Onboarding simples e visual · Filtros úteis de dieta/orçamento · Boa curadoria de pratos	Não busca restaurante real, para no "quero isso" · Usuário ainda precisa abrir outro app pra pedir · Sem modo grupo
imdecisive	Sessão em grupo por código, swipe por categoria de refeição (brunch, jantar, sobremesa, drinks).	Multiplayer nativo desde o início · Convite simples por código de sessão	Só EUA/Coreia, sem foco Brasil · Sem integração com restaurante real · Dev solo, tração praticamente nula
BiteSwipe (CleverCraft LLC)	Swipe em grupo (até 10 pessoas), match quando todos concordam num restaurante real, com filtros de cozinha/preço/nota/raio.	Já fecha o loop em restaurante real · Filtros robustos (preço, nota, raio, aberto agora) · Monetização (assinatura) já testada	Não converge por tipo de comida antes, começa direto no restaurante · Tração ínfima (poucas dezenas de downloads) · Sem foco em mercado local
BiteSwipe (Allset Technologies)	Mesmo conceito de match em grupo por restaurante real; concorrente direto do anterior, com nome idêntico.	Confirma validação de mercado (duas equipes independentes com a mesma ideia)	Sem avaliações públicas suficientes · Nome duplicado no nicho gera confusão pro usuário
Food Swipe — Find Restaurants	Sessão compartilhada de swipe entre amigos até bater o match num restaurante.	Proposta simples e direta · Foco social claro	Sem convergência de tipo de comida antes do swipe · App novo, sem histórico de avaliação

Nota: números de download/avaliação não são expostos oficialmente pelas lojas; os valores citados vêm de agregadores de terceiros e da ausência de avaliações visíveis nas próprias fichas dos apps.

3. Diferenciais do projeto frente à concorrência

- **Fluxo unificado:** convergência de gosto (swipe amplo → específico) e descoberta de restaurante real no mesmo fluxo, sem trocar de app no meio do caminho. Nenhum concorrente listado faz as duas coisas juntas.
- **Foco inicial no mercado brasileiro:** nenhum concorrente direto tem presença, catálogo ou dados de comportamento validados no Brasil.
- **Onboarding sem fricção:** início de sessão sem cadastro obrigatório (autenticação anônima), reduzindo a barreira de entrada que apps como o imdecisive e o BiteSwipe não eliminam completamente.
- **Arquitetura pronta para evoluir:** MVVM + Firebase desde o MVP permite adicionar sessão em grupo (o recurso mais copiado no nicho) sem redesenhar o backend, em vez de travar essa funcionalidade desde o dia um.
- **Base de dados real de restaurantes:** recomendação via Google Places API (endereço, distância, avaliações reais), e não uma lista estática de pratos. É um diferencial direto sobre WhatToEat e imdecisive.

4. Especificação técnica do projeto

4.1 Stack de tecnologias

Camada	Tecnologia	Função no projeto
Cliente mobile	Flutter (Dart)	Base de código única para iOS e Android
Gerenciamento de estado	Riverpod	Implementa a camada ViewModel do MVVM (providers/notifiers)
Backend / dados	Firebase Firestore	Persistência de sessões, histórico de escolhas e preferências
Autenticação	Firebase Authentication (modo anônimo)	Identifica sessão sem exigir cadastro do usuário
Busca de restaurantes	Google Places API	Retorna restaurantes reais próximos por tipo de comida
Evolução futura	Firebase Cloud Functions	Orquestração de sessões multiplayer em tempo real (modo grupo)

4.2 Arquitetura MVVM (Flutter)

O app segue o padrão MVVM (Model – View – ViewModel), com uma camada adicional de Repository/Service para isolar o acesso a dados externos:

- **View:** widgets Flutter responsáveis apenas por renderizar a UI e capturar interação (tela de swipe, tela de resultado convergido, tela de detalhe do restaurante). Não contém lógica de negócio; apenas observa o estado exposto pela ViewModel.
- **ViewModel:** Notifiers/Providers Riverpod que guardam o estado da sessão (categoria atual, histórico de swipes, resultado convergido) e expõem métodos que a View chama em resposta a eventos de UI. Traduz interação em chamadas ao Repository e re-expõe o resultado como estado observável.
- **Model:** classes de dados imutáveis (ex.: FoodCategory, SwipeChoice, Session, Restaurant), sem qualquer dependência de Flutter ou widgets, testáveis isoladamente.
- **Repository / Service:** camada que abstrai a origem dos dados (FirestoreRepository para o Firebase, PlacesService para o Google Places API), de forma que a ViewModel nunca fale diretamente com Firebase ou HTTP.

4.3 Fluxo do produto (MVP)

1. Usuário abre o app e inicia uma sessão de swipe, sem necessidade de cadastro.
2. A primeira decisão é ampla (ex.: salgado vs. doce); cada swipe seguinte afunila a categoria (ex.: salgado → massa → macarrão).

3. Ao convergir num tipo de comida específico, o app consulta o Google Places API por restaurantes próximos que atendem esse tipo.
4. O usuário recebe uma recomendação de restaurantes reais (nome, distância, avaliação), prontos para abrir no delivery de preferência.

4.4 Escopo: MVP vs. evolução futura

MVP (fase 1)	Evolução futura
Sessão solo, sem cadastro	Sessão em grupo (múltiplos usuários votando na mesma sessão, via Firebase)
Convergência de gosto por swipe binário	Histórico de sessões e favoritos por usuário
Recomendação de restaurante via Google Places	Filtros avançados (preço, dieta, distância) e notificações de novidades

5. Termo de compromisso

Este documento formaliza o escopo, a arquitetura, os diferenciais de mercado e o cronograma levantados para o projeto descrito acima, e serve como referência de compromisso para o início da construção do aplicativo. Alterações relevantes de escopo, arquitetura ou cronograma devem ser refletidas em uma nova versão deste documento.

6. Cronograma e organização do time

Seção adicionada na versão 1.1.

6.1 Calendário da disciplina

Aula	Data	Conteúdo previsto
01	06/08	Contexto e simulação profissional
02	13/08	Formação das equipes e integração (aula conjunta)
03	20/08	Ideação e Design Thinking (aula conjunta)
04	27/08	Benchmarking, visão do produto e definição do MVP
05	03/09	Backlog e modelagem inicial
06	10/09	Arquitetura e planejamento da Sprint 1 (aula conjunta)
07	17/09	Sprint 1: construção do primeiro incremento
08	24/09	Review e retrospectiva da Sprint 1
09	01/10	Sprint 2: evolução do fluxo principal
10	08/10	Review e retrospectiva da Sprint 2
11	15/10	Sprint 3: consolidação para o seminário
12	22/10	Seminário de andamento e Review da Sprint 3 (aula conjunta)
13	29/10	Sprint 4: evolução orientada por feedback
14	05/11	Review da Sprint 4, estratégia e validação (aula conjunta)
15	12/11	Sprint 5: consolidação funcional e técnica
16	19/11	Review e retrospectiva da Sprint 5
17	26/11	Sprint 6: estabilização e preparação do pitch
18	03/12	Apresentação final e encerramento da Sprint 6 (aula conjunta)
19	10/12	Feedback 360° e encerramento avaliativo
20	17/12	Fechamento dos conceitos e do diário de classe

6.2 Sprints e metas

Sprints de duas semanas, sempre encerradas na aula de review, onde acontecem a review, a retrospectiva e o planejamento da sprint seguinte.

Sprint	Período	Desenvolvimento	Marketing e divulgação
1	10/09 → 24/09	Repositório, gestão do backlog, arquitetura e processo	Definição das frentes e papéis
2	24/09 → 08/10	Firebase e sessão anônima; árvore de categorias; swipe convergente	Posicionamento, persona, identidade visual, perfis nas redes
3	08/10 → 22/10	Localização, Google Places e lista de restaurantes; fluxo completo para o seminário	Landing page e material do seminário
4	22/10 → 05/11	Detalhe do restaurante, abrir no delivery, persistência da sessão; ajustes do feedback	Pesquisa e teste de usabilidade com usuários
5	05/11 → 19/11	Estados de erro, regras de segurança do Firestore, cobertura de testes	Campanha de pré-lançamento
6	19/11 → 03/12	Estabilização e build de release	Pitch final, vídeo demo, material da loja

6.3 Frentes do time

O time trabalha em frentes que compartilham o mesmo backlog e as mesmas sprints:

- **Desenvolvimento:** construção do aplicativo, testes, integração contínua e especificações técnicas (Spec-Driven Development).
- **Marketing e divulgação:** posicionamento, identidade visual, redes sociais, landing page, pesquisa e validação com usuários, seminário e pitch.
- **Produto/UX:** priorização do backlog, protótipos e critérios de aceitação (pode ser acumulada por integrantes das outras frentes).

6.4 Gestão e acompanhamento

- **Backlog:** versionado no repositório (`.github/backlog/backlog.yml`) e sincronizado automaticamente como Issues do GitHub, com uma milestone por sprint.
- **Quadro:** GitHub Projects com status (Backlog, Todo, In Progress, In Review, Done), sprint e frente de cada item, atualizado automaticamente por GitHub Actions.
- **Acompanhamento:** relatório automático semanal por sprint e frente, usado como roteiro da review.
- **Qualidade:** todo código entra por pull request revisado por outro integrante, com integração contínua (formatação, análise estática e testes).
- **Documentação:** fonte canônica em `docs/contexto.md`; cada funcionalidade relevante tem especificação (`docs/specs/`) antes da implementação.

Antônio Cláudio Costa Junior — Desenvolvedor responsável

Data